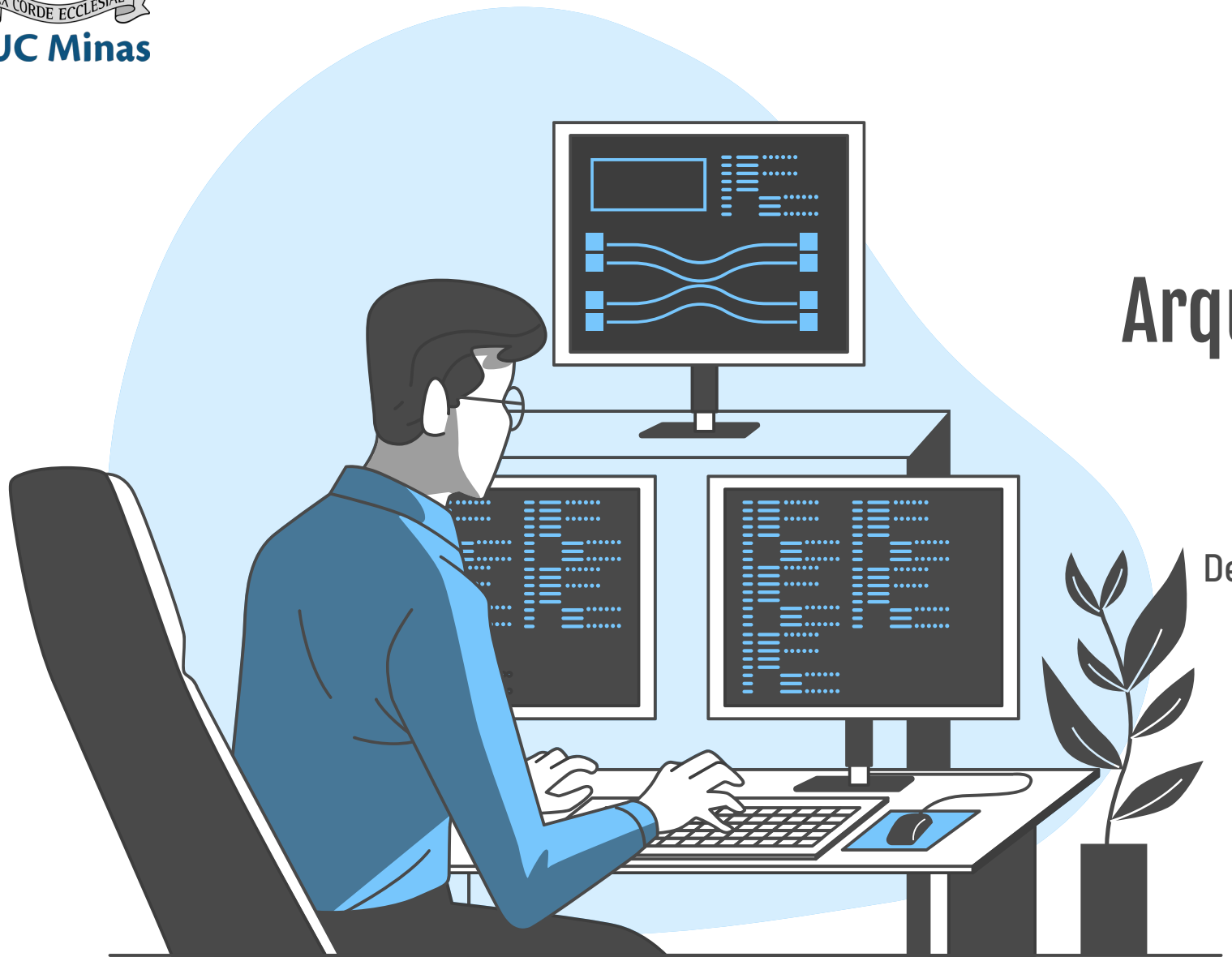
PUC Minas



Aula 01 · FullStack

Arquitetura full stack: como front e back conversam

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Descrever as três partes de uma aplicação full stack: **front**, **back** e **banco**

2

Entender **origem**, **porta** e por que `5173` e `8080` são servidores diferentes

3

Ler uma requisição e uma resposta HTTP com JSON

4

Conhecer o **contrato da API** entre as duas equipes

5

Comparar as arquiteturas: Thymeleaf, SPA separada e SPA servida pelo Spring

Antes de começar

- Pasta Back-end (aulas 01 a 07)
- Pasta Front-end (aulas 01 a 07)

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 Até agora, cada um no seu canto

02 Origem: protocolo + domínio + porta

03 O que trafega

04 O contrato da API

05 Três formas de montar uma aplicação web

06 Aprofundando: o caminho de um clique

01

Até agora, cada um no seu canto

Até agora, cada um no seu canto

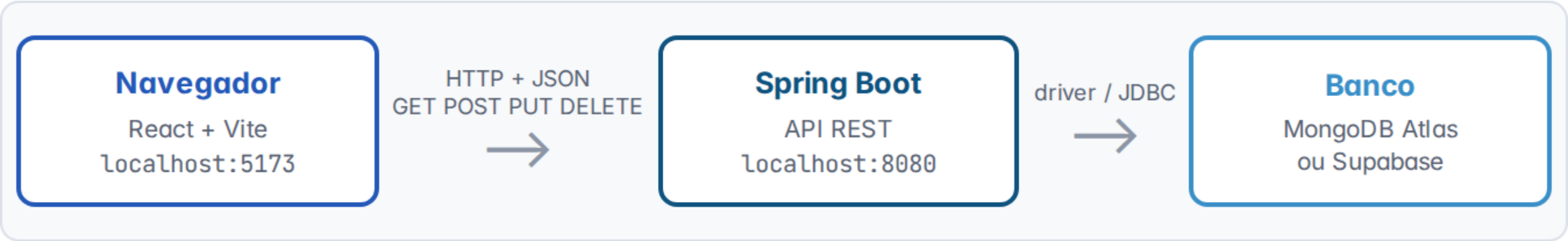
Na pasta Back-end você fez APIs Spring Boot testadas pelo cliente HTTP. Na pasta Front-end, telas React que consumiam APIs públicas (ViaCEP, PokeAPI) ou o Supabase. Agora as duas metades vão **rodar ao mesmo tempo e conversar**: o React manda requisições para o **seu** Spring Boot, que lê e grava no banco.

01

Até agora, cada um no seu canto

Aula 01 · FullStack

Até agora, cada um no seu canto



Arquitetura do projeto `Exemplo_Integracao`: três peças, duas conversas.

Até agora, cada um no seu canto

Peça	Roda em	Durante o desenvolvimento	Em produção
Front (React + Vite)	Navegador do usuário	`npm run dev` → `http://localhost:5173`	Vercel → `https://app.vercel.app`
Back (Spring Boot)	Servidor	`./mvnw spring-boot:run` → `http://localhost:8080`	Render → `https://api.onrender.com`
Banco	Nuvem	MongoDB Atlas / Supabase	Os mesmos

02

Origem: protocolo + domínio + porta

Origem: protocolo + domínio + porta

Para o navegador, uma **origem** é a combinação de protocolo, domínio e porta. Basta um deles mudar para ser outra origem:

URL	Origem	Mesma origem do front?
`http://localhost:5173/alunos`	`http://localhost:5173`	É o front
`http://localhost:8080/api/alunos`	`http://localhost:8080`	**Não** (porta diferente)
`https://app.vercel.app`	`https://app.vercel.app`	É o front em produção
`https://api.onrender.com/api/alunos`	`https://api.onrender.com`	**Não** (domínio diferente)

Origem: protocolo + domínio + porta

Por isso o back-end precisa **autorizar** a origem do front (CORS, aula 03). Esse é o ponto que mais trava quem integra pela primeira vez.

02

Origem: protocolo + domínio + porta

Aula 01 · FullStack

03

O que trafega

O que trafega

Toda conversa é uma **requisição** (do React) e uma **resposta** (do Spring), em texto:

Requisição (React → Spring)

```
POST /api/alunos HTTP/1.1
Host: localhost:8080
Origin: http://localhost:5173
Content-Type: application/json

{"nome":"Ana","email":"ana@email.com",
 "curso":"Engenharia de Software"}
```

Resposta (Spring → React)

```
HTTP/1.1 201 Created
Access-Control-Allow-Origin: http://localhost:5173
Content-Type: application/json

{"id":"6714c0f2a1b3c45d6e7f8091",
 "nome":"Ana","email":"ana@email.com",
 "curso":"Engenharia de Software"}
```

04

O contrato da API

O contrato da API

Front e back precisam concordar sobre **endpoints**, **formato do JSON** e **formato dos erros**. Escreva esse contrato no README antes de começar, como no projeto `Exemplo_Integracao`:

Método	Endpoint	Corpo enviado	Resposta
`GET`	`/api/status`		`{ "backend": "online", "mongodb": "online" }`
`GET`	`/api/alunos?nome=ana`		`200` + lista
`POST`	`/api/alunos`	`{ nome, email, curso }`	`201` + aluno com `id`
`PUT`	`/api/alunos/{id}`	`{ nome, email, curso }`	`200` + aluno
`DELETE`	`/api/alunos/{id}`		`204`
erro	qualquer		`{ status, erro, mensagem, caminho, timestamp }`

O contrato da API

O campo `mensagem` do erro é o que o React mostra para o usuário. Assim, a regra “e-mail inválido” fica **no back-end** e o front só exibe.

04

O contrato da API

Aula 01 · FullStack

05

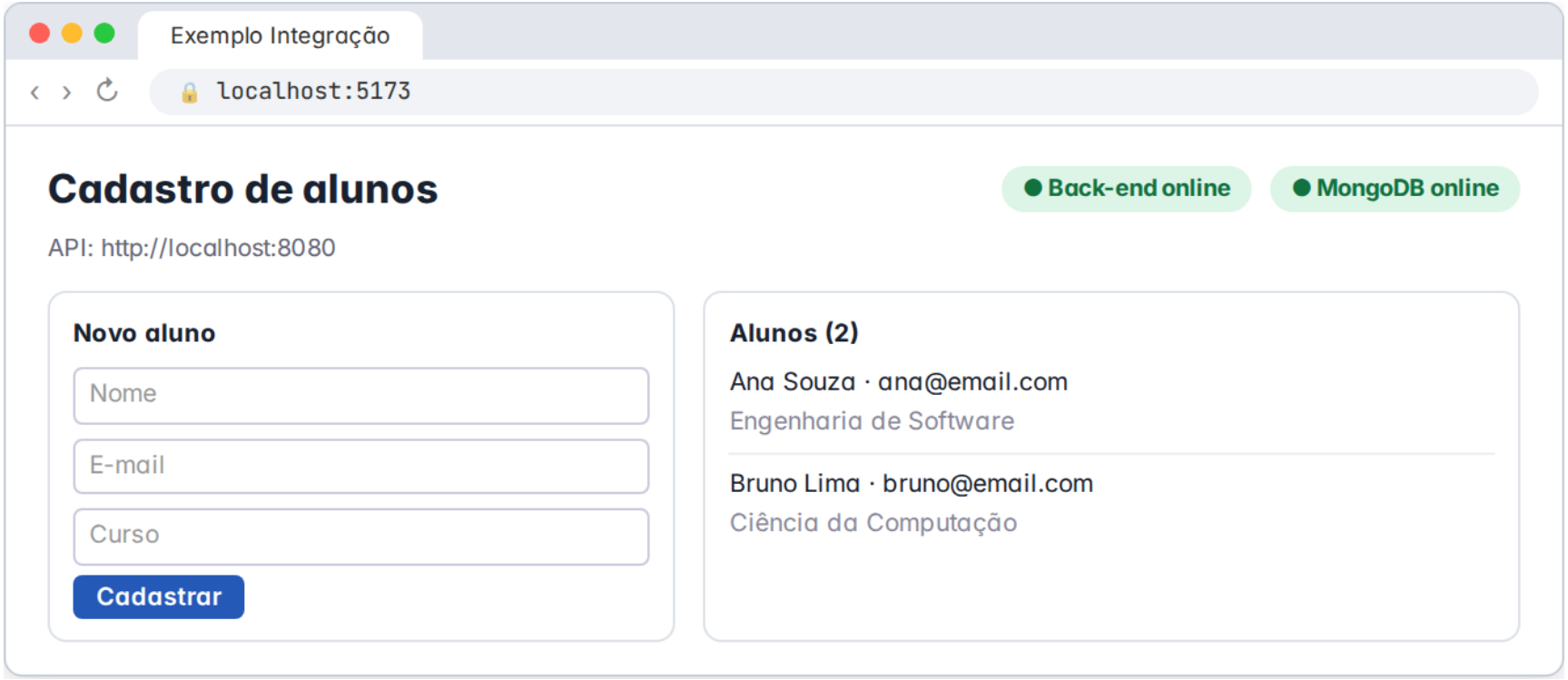
Três formas de montar uma aplicação web

Três formas de montar uma aplicação web

Arquitetura	Como funciona	Projetos	Deploy
Server-side (Thymeleaf)	O Spring gera o HTML	`SecureLogin`, `SecureLoginPUC_3`	Só Render
SPA separada	React e Spring são projetos e servidores diferentes	`Exemplo_Integracao`, `AraCoffee`, `PalpiteDTI`, `DronesDTI`	Vercel + Render
SPA dentro do Spring	O build do React vai para `static/` do Spring	Aula 07 desta pasta	Só Render

Três formas de montar uma aplicação web

A segunda é a mais comum no mercado e a que mais usamos: cada parte escala e é publicada de forma independente. A terceira é útil quando você quer **um único deploy**.



Front do `Exemplo_Integracao` conectado: o painel no topo consulta `GET /api/status` para mostrar se o back e o banco respondem.

06

Aprofundando: o caminho de um clique

Aprofundando: o caminho de um clique

1. O usuário clica em **Cadastrar** no React; o `onSubmit` chama `criarAluno(dados)`.
2. O `httpClient` monta a URL com a `BASE_URL` e chama `fetch` com método POST e corpo JSON.
3. O navegador envia um *preflight* `OPTIONS` (origem diferente) e, autorizado, o `POST`.
4. O Spring converte o JSON em objeto, o controller chama o service, o service valida e chama o repository.
5. O banco grava; a resposta `201` com o JSON do aluno volta pelo mesmo caminho e o React atualiza a lista.

Aprofundando: o caminho de um clique

Camada	Tecnologia	Arquivo típico
Tela	React	`components/AlunoForm.jsx`
Comunicação (front)	fetch	`services/alunoService.js`, `httpClient.js`
Entrada HTTP (back)	Spring MVC	`controller/AlunoController.java`
Regras	Java	`service/AlunoService.java`
Dados	Spring Data	`repository/AlunoRepository.java`
Persistência	MongoDB Atlas / Supabase	coleção `alunos` / tabela `aluno`

JSON: o idioma comum

Objetos

`{ "nome": "Ana", "ativo": true }` ↔ classe/record Java ↔ objeto JS.

Listas

`[ {...}, {...} ]` ↔ `List` ↔ array.

Datas

Viajam como texto ISO: `"2026-11-03T13:05:00Z"`. Converta no front com `new Date(...)`.

Números

`BigDecimal` vira número; para dinheiro, formate no front com `toLocaleString("pt-BR")`.

Monolito x front e back separados

Monolito (Thymeleaf)

- Um projeto, um deploy
- Sem CORS
- A tela depende do servidor a cada página

vs

SPA + API

- Dois projetos, dois deploys
- Precisa de CORS e de contrato
- A mesma API pode servir web, app mobile e outros sistemas

Aprofundando: o caminho de um clique

Contrato espelhado nos dois lados

```
// front: o que o React espera
{ "id": "6714...", "nome": "Ana", "email": "ana@email.com", "curso": "ES" }

// back: o record que produz exatamente isso
public record AlunoResponseDTO(String id, String nome, String email, String curso) { }
```

Aprofundando: o caminho de um clique

Projeto FullStack	Banco	Destaques
`Exemplo_Integracao`	MongoDB Atlas	CRUD mínimo, painel de status, testes
`AraCoffee`	MongoDB Atlas	Cardápio com categorias e carga inicial
`PalpiteDTI`	MongoDB	Jogo, ranking, Docker, k6, Actuator
`DronesDTI`	MongoDB	Simulação, Dockerfiles de front e back, testes de carga

Aprofundando: o caminho de um clique

Dica

Ao começar um projeto full stack, escreva primeiro o contrato da API e combine com quem faz o front. Os dois lados podem trabalhar em paralelo: o front usa dados falsos com o mesmo formato até o back ficar pronto.

07

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

Para o navegador, `http://localhost:5173` e `http://localhost:8080` são:

A. A mesma origem

B. Origens diferentes (porta diferente)

C. O mesmo servidor

D. Inválidas

Quiz · questão 1 de 5

Resposta

Para o navegador, `http://localhost:5173` e `http://localhost:8080` são:

A. A mesma origem

✓ Origens diferentes (porta diferente)

C. O mesmo servidor

D. Inválidas

Por quê: Origem = protocolo + domínio + porta.

Pergunta

O que é o contrato da API?

A. O contrato de hospedagem

B. O acordo sobre endpoints, formato do JSON e dos erros

C. O arquivo pom.xml

D. A licença

Quiz · questão 2 de 5

Resposta

O que é o contrato da API?

A. O contrato de hospedagem

✓ O acordo sobre endpoints, formato do JSON e dos erros

C. O arquivo pom.xml

D. A licença

Por quê: Escreva antes de programar.

Pergunta

Onde deve ficar a regra “e-mail válido”?

A Só no front

B No back (o front pode repetir para UX)

C No CSS

D No banco apenas

Resposta

Onde deve ficar a regra “e-mail válido”?

A Só no front

✓ No back (o front pode repetir para UX)

C No CSS

D No banco apenas

Por quê: O front só exibe a mensagem do back.

Pergunta

Qual arquitetura publica front e back separados?

A Thymeleaf

B SPA separada (Vercel + Render)

C Monolito

D Nenhuma

Resposta

Qual arquitetura publica front e back separados?

A Thymeleaf

✓ SPA separada (Vercel + Render)

C Monolito

D Nenhuma

Por quê: É a mais usada nos projetos FullStack.

Pergunta

Em que formato front e back trocam dados?

A XML

B JSON sobre HTTP

C CSV

D Binário

Resposta

Em que formato front e back trocam dados?

A XML

✓ JSON sobre HTTP

C CSV

D Binário

Por quê: O Jackson converte no back; o fetch/JSON no front.

Fechamento

Resumo da aula

- ✓ Full stack = **front** (navegador) + **back** (servidor) + **banco**, cada um em seu lugar.
- ✓ Origem = protocolo + domínio + porta; 5173 e 8080 são origens diferentes, daí o CORS.
- ✓ A conversa é HTTP com JSON; escreva o **contrato da API** antes de programar.
- ✓ Erros padronizados com `mensagem` deixam as regras no back e a exibição no front.
- ✓ Três arquiteturas: Thymeleaf, SPA separada (mais usada) e SPA servida pelo Spring.

Exercícios

1. Desenhe a arquitetura de um sistema de biblioteca com as URLs de desenvolvimento e produção.

2. Escreva o contrato da API (tabela de endpoints e formato de erro) para o CRUD de livros.

3. Abra o DevTools no front do `Exemplo_Integracao` e identifique a requisição, os cabeçalhos `Origin` e `Access-Control-Allow-Origin`.

4. Liste vantagens e desvantagens de publicar front e back separados.

Para ir além

- Projetos `PROJETOS/FullStack/*` (leia os README de cada um)

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 01?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web